Modul 3.3: Besitz und Privilegien

Anschluss an Modul 3.2

In Modul 3.2 hast du Berechtigungen mit `chmod` gesetzt. Jetzt untersuchst und änderst du den **Besitz** von Dateien: den Benutzer, dem eine Datei gehört, und ihre Gruppe. Dafür lernst du `chgrp`, `chown` und `sudo`.

> **Caution**
>
> Die Übung verwendet nur das vorbereitete Projekt unter `~/projekt/`. Verändere keine Dateien außerhalb dieses Pfads. Prüfe vor jedem privilegierten Befehl Zielpfad und Zielbesitzer; verwende `sudo` nur bei den ausdrücklich genannten Befehlen.

1. Wem gehört was?

Zeige Besitzer und Gruppe aller Projektdateien:

```
ls -l ~/projekt/
```

Eine Zeile wie diese enthält neben den Rechten auch Besitzer und Gruppe:

```
-rw-r--r-- 1 anna entwicklung 1024 Mar 15 10:00 datei.txt
             ^^^^ ^^^^^^^^^^^
             User  Group
```

Jede Datei hat genau einen Besitzer und genau eine Gruppe. Die drei Berechtigungsblöcke aus dem letzten Lab gelten für Besitzer, Gruppe und Andere. Einige Dateien gehören `student:student`, andere `root:root`.

Beachte besonders `datenbank.conf`. Sie gehört `root:root` und hat den Modus `600`. Deshalb darf nur root sie lesen.

2. Gruppe mit chgrp ändern

Prüfe deine Gruppen:

```
groups
```

`team-config.txt` gehört dir und der Gruppe `student`. Die Gruppe `entwicklung` passt besser zum gemeinsamen Projekt. `chgrp` (**change group**) ändert die Gruppenzuordnung:

```
chgrp NEUE_GRUPPE DATEI
```

Du darfst die Gruppe einer eigenen Datei ändern, wenn du Mitglied der Zielgruppe bist. Ändere die Gruppe:

```
chgrp entwicklung ~/projekt/team-config.txt
```

Prüfe:

```
ls -l ~/projekt/team-config.txt
```

Die Gruppe ist jetzt `entwicklung`. Ob Teammitglieder tatsächlich lesen können, hängt zusätzlich vom Gruppen-Berechtigungsblock ab.

3. Besitzer ändern: Permission denied

`mein-script.sh` gehört dir. Versuche, den Besitzer an Anna zu übertragen:

```
chown anna ~/projekt/mein-script.sh
```

Das scheitert mit `Operation not permitted`. `chown` (**change owner**) darf den Besitzer nicht durch einen normalen Benutzer ändern. Auch der aktuelle Besitzer darf eine Datei nicht einfach an eine andere Person übertragen.

Das schützt vor unkontrollierten Besitzwechseln und verhindert, dass Dateien mit besonderen Rechten beliebig an andere Konten weitergegeben werden.

4. sudo: einen Befehl als root ausführen

`sudo` (**substitute user do**) verleiht nur dem folgenden Befehl erhöhte Rechte. Du wirst nicht dauerhaft root:

```
sudo -l
```

Im Labor ist `sudo` ohne Passwort konfiguriert. Auf echten Servern ist die Regel normalerweise eingeschränkter und kann ein Passwort verlangen.

Teste einen einzelnen Befehl:

```
sudo whoami
whoami
```

Die erste Ausgabe ist `root`, die zweite wieder `student`.

5. Mit sudo chown den Besitzer ändern

Ändere den Besitzer von `mein-script.sh`:

```
sudo chown anna ~/projekt/mein-script.sh
```

Prüfe:

```
ls -l ~/projekt/mein-script.sh
```

Die Gruppe bleibt unverändert, weil `chown` ohne `:gruppe` nur den Besitzer ändert. Die Datei gehört dir danach nicht mehr.

6. Besitzer und Gruppe gleichzeitig ändern

Mit `user:group` ändert `chown` beide Werte in einem Befehl. Setze `team-config.txt` auf Besitzer `tom` und Gruppe `entwicklung`:

```
sudo chown tom:entwicklung ~/projekt/team-config.txt
```

Prüfe:

```
ls -l ~/projekt/team-config.txt
```

Der Doppelpunkt trennt Besitzer und Gruppe. `chown :gruppe datei` kann auch nur die Gruppe ändern, für eine eigene Datei ist `chgrp` jedoch meist passender und benötigt kein `sudo`.

7. Root-Dateien übernehmen oder einmalig lesen

Prüfe zunächst den Zustand:

```
ls -l ~/projekt/server-config.txt ~/projekt/datenbank.conf
```

Übernimm `server-config.txt`:

```
sudo chown student:student ~/projekt/server-config.txt
```

Prüfe:

```
ls -l ~/projekt/server-config.txt
```

Für `datenbank.conf` musst du den Besitz nicht ändern, wenn du nur einmalig lesen möchtest:

```
sudo cat ~/projekt/datenbank.conf
```

`sudo` gilt hier nur für `cat`. Die Datei bleibt `root:root`.

8. Einen ganzen Verzeichnisbaum ändern

`~/projekt/shared/` gehört anfangs `root:entwicklung`. Der gesamte Baum soll dem Benutzer `student` und der Gruppe `entwicklung` gehören.

Prüfe zuerst:

```
ls -la ~/projekt/shared/
```

Verwende anschließend den ausdrücklich begrenzten rekursiven Befehl:

```
sudo chown -R student:entwicklung ~/projekt/shared/
```

Prüfe danach:

```
ls -la ~/projekt/shared/
```

`-R` bedeutet rekursiv: Der Befehl gilt für das Verzeichnis und seinen gesamten Inhalt. Prüfe den Zielpfad vor der Ausführung besonders sorgfältig.

9. Dateien für das Team vorbereiten

`datenbank.conf` enthält in dieser Übung nur Beispieldaten. Der aktuelle Zustand ist `root:root` mit Modus `600`. Anna soll die Datei verwalten, die Gruppe `entwicklung` soll sie lesen können.

Setze Besitzer und Gruppe:

```
sudo chown anna:entwicklung ~/projekt/datenbank.conf
```

Prüfe:

```
ls -l ~/projekt/datenbank.conf
```

Besitz allein gewährt der Gruppe noch keinen Zugriff. Wenn der Modus weiterhin `600` ist, muss für das ausdrücklich beschriebene Ziel zusätzlich das Gruppenleserecht gesetzt werden:

```
sudo chmod 640 ~/projekt/datenbank.conf
```

Prüfe den vollständigen Zielzustand:

```
ls -l ~/projekt/datenbank.conf
```

Erwartet wird `anna entwicklung` und `-rw-r-----`. Damit darf Anna lesen und schreiben, Mitglieder von `entwicklung` dürfen lesen, Andere haben keinen Zugriff.

Sicherheitsregeln

- Schreibende Änderungen bleiben unter `~/projekt/`.
- Prüfe jeden vollständigen Zielpfad vor `sudo chown` und `sudo chmod`.
- Verwende niemals `sudo chown -R` auf `/`, `/home` oder einem unklaren Pfad.
- `sudo` verleiht nur dem unmittelbar folgenden Befehl Rechte.
- Besitzerwechsel geben den bisherigen Besitz ab und können den Zugriff verändern.
- Nutze für reine Gruppenänderungen eigener Dateien bevorzugt `chgrp`.

Zusammenfassung

Du kannst jetzt:

- Besitzer und Gruppe mit `ls -l` untersuchen,
- eine eigene Datei mit `chgrp` einer passenden Gruppe zuordnen,
- erklären, warum normales `chown` scheitert,
- mit `sudo` einen einzelnen Befehl als root ausführen,
- Besitzer und Gruppe mit `chown user:group` setzen,
- einen begrenzten Verzeichnisbaum mit `chown -R` ändern,
- Besitz und Berechtigungen gemeinsam auf einen fachlichen Zielzustand abstimmen.

Im nächsten Modul vergleichst du komplexere Zugriffssituationen und die Auswirkungen von Besitz, Gruppe und Rechten gemeinsam.